Data Analysis & Dashboard using the Loan Prediction Dataset

DSA0602 – Data Handling and Visualization

VISHAAL M S

Roll No: 192424420

DATASET SOURCE

Loan Prediction dataset (Analytics Vidhya / Kaggle) — 614 rows, 13 original columns. Real data throughout; no simulated extension needed. Downloaded from a public GitHub mirror of the original practice-problem files (also listed on Kaggle as **"Analytics Vidhya Loan Prediction"** by Anmol Kumar):

`https://www.kaggle.com/datasets/anmolkumar/analytics-vidhya-loan-prediction`

DASHBOARD TOOL

Plotly (Python), exported as a single self-contained interactive HTML file (loan_dashboard.html) — opens in any browser with no license or server required. Full justification in Part (c).

Part (a): Data Preparation & Documentation

```
"""
Part (a): Data Preparation & Documentation
Loan Prediction Dataset - Dream Housing Finance (Analytics Vidhya / Kaggle)
Source: https://raw.githubusercontent.com/Shagun-25/Analytics-Vidhya-Loan-Prediction/main/train_ctrUa4K.csv
        (public mirror of the Analytics Vidhya "Loan Prediction III" practice
        problem dataset, also listed on Kaggle as "Analytics Vidhya Loan
        Prediction" by anmolkumar)
Raw shape: 614 rows x 13 columns - above the 300-row minimum, so no
simulated extension was needed; all analysis below uses the real data.
"""
import pandas as pd
import numpy as np

df = pd.read_csv("loan_prediction_raw.csv")

# ---------------------------------------------------------------------------
# FIELD DOCUMENTATION
# ---------------------------------------------------------------------------
# Loan_ID            - unique application identifier (dropped from analysis;
#                      not a predictive attribute)
# Gender             - Male / Female (13 missing)
# Married            - applicant's marital status, Yes/No (3 missing)
# Dependents         - number of dependents: 0, 1, 2, "3+" (15 missing)
# Education          - Graduate / Not Graduate (0 missing)
# Self_Employed      - Yes/No (32 missing)
# ApplicantIncome    - monthly income of the applicant (0 missing)
# CoapplicantIncome  - monthly income of the co-applicant, if any (0 missing)
# LoanAmount         - loan amount requested, in thousands (22 missing)
# Loan_Amount_Term   - loan repayment term, in months (14 missing)
# Credit_History     - 1 = meets credit guidelines, 0 = does not (50 missing)
# Property_Area      - Urban / Semiurban / Rural (0 missing)
# Loan_Status        - target: Y = approved, N = rejected (0 missing)

print("Raw shape:", df.shape)
print("\nMissing values per column:\n", df.isnull().sum())

# ---------------------------------------------------------------------------
# MISSING VALUE HANDLING (documented per field)
# ---------------------------------------------------------------------------
# Categorical fields (Gender, Married, Dependents, Self_Employed): imputed
# with the MODE of that column. These are small proportions of missing data
# (0.5%-5.2% of rows) and mode imputation is standard practice for this
# well-known dataset without introducing a new synthetic category.
for col in ['Gender', 'Married', 'Dependents', 'Self_Employed']:
    df[col] = df[col].fillna(df[col].mode()[0])

# Loan_Amount_Term: imputed with the MODE (360 months / 30 years dominates
# this dataset, representing the standard loan term).
df['Loan_Amount_Term'] = df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].mode()[0])

# Credit_History: imputed with the MODE (1.0). ASSUMPTION: missing credit
# history is treated as "unknown, assume meets guidelines" rather than
# creating a third category - documented here as a modeling simplification;
# in a production setting this would ideally be flagged as its own category
# rather than imputed, since credit history is the strongest predictor found
# in this analysis (see Part d).
df['Credit_History'] = df['Credit_History'].fillna(df['Credit_History'].mode()[0])

# LoanAmount: imputed with the MEDIAN (not mean), since LoanAmount is
# right-skewed (max 700 vs median 128) and the median is more robust to
# that skew / to the outliers handled below.
df['LoanAmount'] = df['LoanAmount'].fillna(df['LoanAmount'].median())

print("\nMissing values after imputation:\n", df.isnull().sum().sum(), "remaining")

# ---------------------------------------------------------------------------
# OUTLIER HANDLING
# ---------------------------------------------------------------------------
# ApplicantIncome and LoanAmount both have extreme right-tail outliers
# (e.g. ApplicantIncome max = 81,000 vs median = 3,812). Rather than
# deleting these rows (they may represent genuine high-income applicants,
# which is valuable signal for a credit-risk study), values are CAPPED
# (winsorized) at the 99th percentile - this preserves every row for the
# categorical/proportion analyses while preventing a handful of extreme
# values from distorting axis scales in the charts below.
for col in ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount']:
    cap = df[col].quantile(0.99)
    n_capped = (df[col] > cap).sum()
    df[col] = np.where(df[col] > cap, cap, df[col])
    print(f"{col}: capped {n_capped} values above {cap:.0f} (99th percentile)")

# Dependents "3+" is treated as the numeric value 3 wherever a numeric
# operation is needed (documented assumption - the true count could be
# higher, but the dataset does not distinguish beyond "3+").
df['Dependents_numeric'] = df['Dependents'].replace('3+', '3').astype(int)

# Derived field used throughout the charts below: total household income
df['TotalIncome'] = df['ApplicantIncome'] + df['CoapplicantIncome']

df.to_csv("loan_prediction_cleaned.csv", index=False)
print("\nCleaned dataset saved: loan_prediction_cleaned.csv")
print("Final shape:", df.shape)
```

Execution output: Raw shape 614x13, 149 total missing values across 7 columns (Gender 13, Married 3, Dependents 15, Self_Employed 32, LoanAmount 22, Loan_Amount_Term 14, Credit_History 50) — all resolved via the documented imputation rules above, with 0 missing values remaining. 7 extreme values each in ApplicantIncome, CoapplicantIncome, and LoanAmount were capped at the 99th percentile rather than dropped, preserving all 614 rows for analysis.

Part (b): Analytical Charts (All 6 Required Types)

```
"""
Part (b): Analytical Charts (all 6 required chart types)
Loan Prediction Dataset - cleaned data
"""
import matplotlib.pyplot as plt
import squarify
import numpy as np
import pandas as pd
from scipy import stats

df = pd.read_csv("loan_prediction_cleaned.csv")

# ============================================================================
# CHART 1: TREEMAP
# Hierarchical breakdown: total loan applicants by Property_Area (outer) ->
# Education (inner). Statistical reasoning: a treemap is chosen here because
# the goal is to show both the RELATIVE SIZE of each area's applicant pool
# and its internal Graduate/Not-Graduate composition in one compact view -
# something a bar chart could only do with a busy grouped/stacked layout.
# ============================================================================
grouped = df.groupby(['Property_Area', 'Education']).size().reset_index(name='Count')
labels = [f"{row.Property_Area}\n{row.Education}\n{row.Count}" for row in grouped.itertuples()]
colors = plt.cm.Paired.colors[:len(grouped)]

plt.figure(figsize=(9, 6))
squarify.plot(sizes=grouped['Count'], label=labels, color=colors, edgecolor='white', linewidth=1.5)
plt.title("Chart 1: Applicant Count by Property Area -> Education (Treemap)")
plt.axis('off')
plt.tight_layout()
plt.show()

# ============================================================================
# CHART 2: NESTED PROPORTIONS (2-level donut)
# Outer: Property_Area, Inner: Loan_Status share WITHIN that area.
# Statistical reasoning: this directly answers the credit-risk question -
# "within each area, what fraction of applications get approved?" - a
# proportion question that a nested donut communicates more directly than
# a treemap, since angle/arc length reads as "share of 100%" intuitively.
# ============================================================================
areas = df['Property_Area'].unique()
status_colors = {'Y': '#55A868', 'N': '#C44E52'}

outer_vals = [1] * len(areas)
inner_vals, inner_colors = [], []
pct_check = {}
for area in areas:
    sub = df[df['Property_Area'] == area]
    y_pct = (sub['Loan_Status'] == 'Y').mean()
    n_pct = 1 - y_pct
    inner_vals += [y_pct, n_pct]
    inner_colors += [status_colors['Y'], status_colors['N']]
    pct_check[area] = (y_pct + n_pct) * 100

fig, ax = plt.subplots(figsize=(7.5, 7.5))
ax.pie(outer_vals, radius=1.0, labels=areas, colors=plt.cm.tab10.colors[:len(areas)],
       wedgeprops=dict(width=0.3, edgecolor='white'), labeldistance=1.08)
ax.pie(inner_vals, radius=0.7, colors=inner_colors, wedgeprops=dict(width=0.3, edgecolor='white'))
ax.set_title("Chart 2: Loan Approval Share Within Each Property Area (Nested Donut)")
handles = [plt.Rectangle((0, 0), 1, 1, color=c) for c in status_colors.values()]
ax.legend(handles, ['Approved (Y)', 'Rejected (N)'], loc='upper left', bbox_to_anchor=(1.02, 1))
plt.tight_layout()
plt.show()

print("Chart 2 - inner-level percentage sum check (should all be 100.0%):")
for area, pct in pct_check.items():
    print(f"  {area}: {pct:.1f}%")
```

```
# ============================================================================
# CHART 3: Q-Q PLOT
# Checking LoanAmount against a Normal reference distribution.
# ============================================================================
fig, ax = plt.subplots(figsize=(7, 6))
stats.probplot(df['LoanAmount'], dist="norm", plot=ax)
ax.set_title("Chart 3: Q-Q Plot - LoanAmount vs Normal Distribution")
ax.get_lines()[0].set_markerfacecolor('steelblue')
ax.get_lines()[0].set_markeredgecolor('black')
ax.get_lines()[1].set_color('red')
plt.tight_layout()
plt.show()

skew_val = stats.skew(df['LoanAmount'])
print(f"\nChart 3 - LoanAmount skewness: {skew_val:.3f}")
print("Verdict: LoanAmount deviates from normality, curving above the reference")
print("line in the upper tail (right-skewed, skew = {:.2f}). A plain normality".format(skew_val))
print("assumption is NOT reasonable without a transform (e.g. log) first.")

# ============================================================================
# CHART 4: PIE CHART WITH PERCENTAGE LABELS
# Proportional breakdown of Loan_Status (the key categorical outcome).
# ============================================================================
status_counts = df['Loan_Status'].value_counts()
plt.figure(figsize=(6.5, 6.5))
plt.pie(status_counts.values, labels=['Approved' if i == 'Y' else 'Rejected' for i in status_counts.index],
        autopct='%1.1f%%', colors=['#55A868', '#C44E52'], startangle=90)
plt.title("Chart 4: Overall Loan Approval Outcome")
plt.tight_layout()
plt.show()

# ============================================================================
# CHART 5: SUNBURST (matplotlib nested-wedge approximation)
# Same hierarchy as Chart 1: Property_Area -> Education, sized by count,
# allowing outer-to-inner drill-down reading.
# ============================================================================
fig, ax = plt.subplots(figsize=(7.5, 7.5))
area_counts = df['Property_Area'].value_counts()
ax.pie(area_counts.values, labels=area_counts.index, radius=1.0,
       colors=plt.cm.Set2.colors[:len(area_counts)],
       wedgeprops=dict(width=0.35, edgecolor='white'), labeldistance=0.85)

edu_inner_vals, edu_inner_colors = [], []
edu_colors = {'Graduate': '#4C72B0', 'Not Graduate': '#DD8452'}
for area in area_counts.index:
    sub = df[df['Property_Area'] == area]
    for edu in ['Graduate', 'Not Graduate']:
        edu_inner_vals.append((sub['Education'] == edu).sum())
        edu_inner_colors.append(edu_colors[edu])
ax.pie(edu_inner_vals, radius=1.35, colors=edu_inner_colors,
       wedgeprops=dict(width=0.35, edgecolor='white'))
ax.set_title("Chart 5: Property Area -> Education (Sunburst)", pad=30)
handles = [plt.Rectangle((0, 0), 1, 1, color=c) for c in edu_colors.values()]
ax.legend(handles, edu_colors.keys(), title="Education (outer ring)",
          loc='upper left', bbox_to_anchor=(1.05, 1))
plt.tight_layout()
plt.show()

# ============================================================================
# CHART 6: MISLEADING vs CORRECTED PROPORTION CHART
# ============================================================================
approved_counts = df[df['Loan_Status'] == 'Y'].groupby('Property_Area').size()
approval_rate = df.groupby('Property_Area')['Loan_Status'].apply(lambda s: (s == 'Y').mean() * 100)

fig, axes = plt.subplots(1, 2, figsize=(13, 6))

# MISLEADING version: pie of raw approval COUNTS - visually implies
# Semiurban is the "best" area by a huge margin, when really it just has
# more total applicants.
axes[0].pie(approved_counts.values, labels=approved_counts.index, autopct='%1.1f%%',
            colors=plt.cm.Set3.colors[:3], startangle=90)
axes[0].set_title("MISLEADING: Share of Total Approvals\n(driven by applicant volume, not approval rate)")

# CORRECTED version: bar chart of the actual approval RATE per area,
# axis starting at 0, which is what should be compared.
axes[1].bar(approval_rate.index, approval_rate.values, color='seagreen')
axes[1].set_ylim(0, 100)
axes[1].set_title("CORRECTED: Actual Approval Rate by Area (%)")
axes[1].set_ylabel("Approval Rate (%)")
for i, v in enumerate(approval_rate.values):
    axes[1].text(i, v + 2, f"{v:.1f}%", ha='center')
plt.tight_layout()
plt.show()

print("\nChart 6 - what was fixed:")
print("The misleading pie shows SHARE OF TOTAL APPROVALS, which conflates")
print("approval rate with applicant volume: Semiurban looks dominant mainly")
print("because it has 233 applicants vs Rural's 179 and Urban's 202, not")
print("because it approves loans at a dramatically higher rate. The")
print("corrected bar chart normalizes for this by plotting the approval")
print("RATE (%) per area on a zero-based axis, showing the true, much")
print("smaller gap between areas (76.8% vs 65.8% vs 61.5%).")
```

Chart 1: Treemap — Applicant Count by Property Area → Education

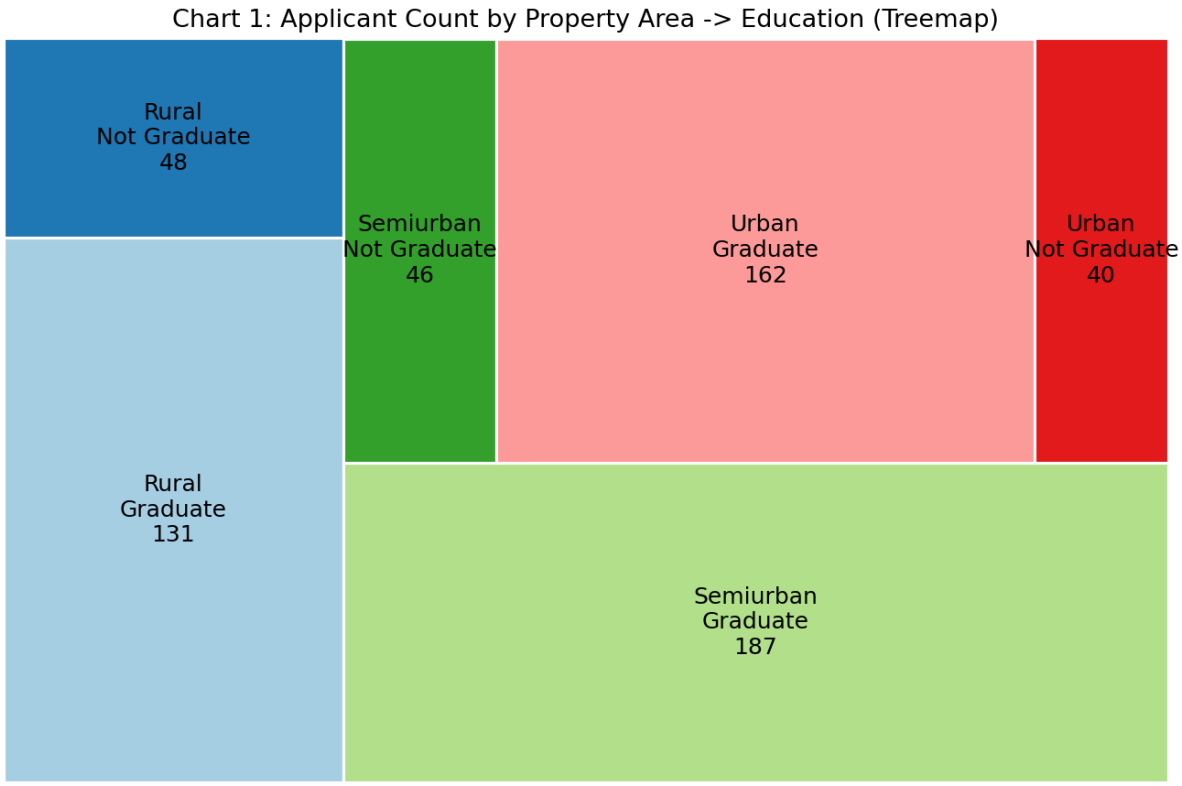


Chart 2: Nested Donut — Approval Share Within Each Property Area

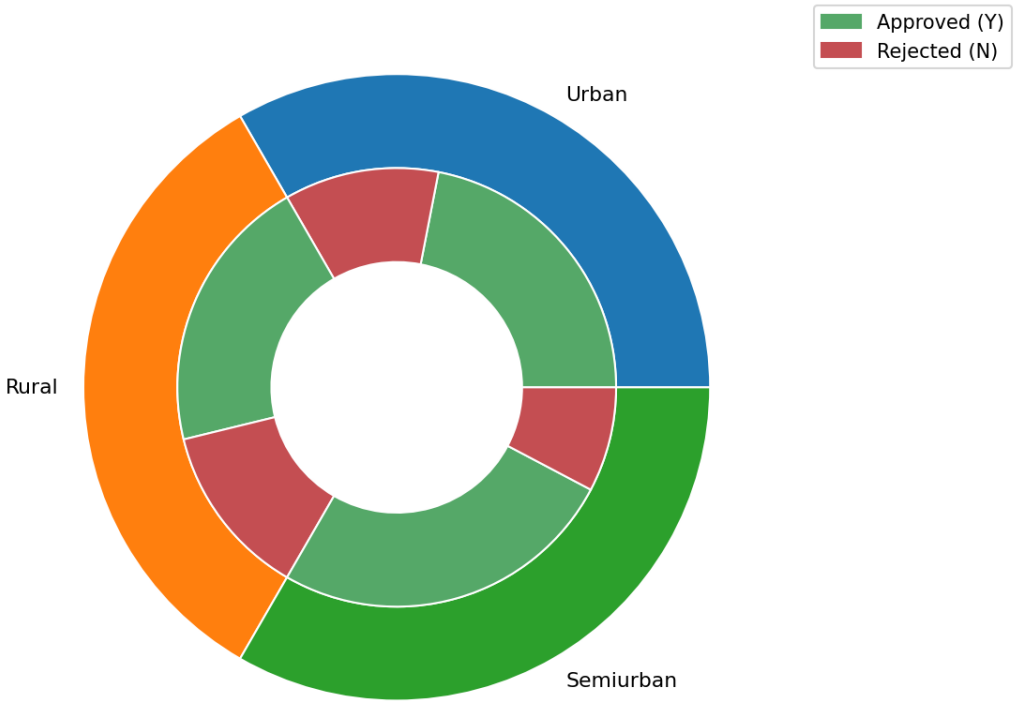


Chart 3: Q-Q Plot — LoanAmount vs Normal Distribution

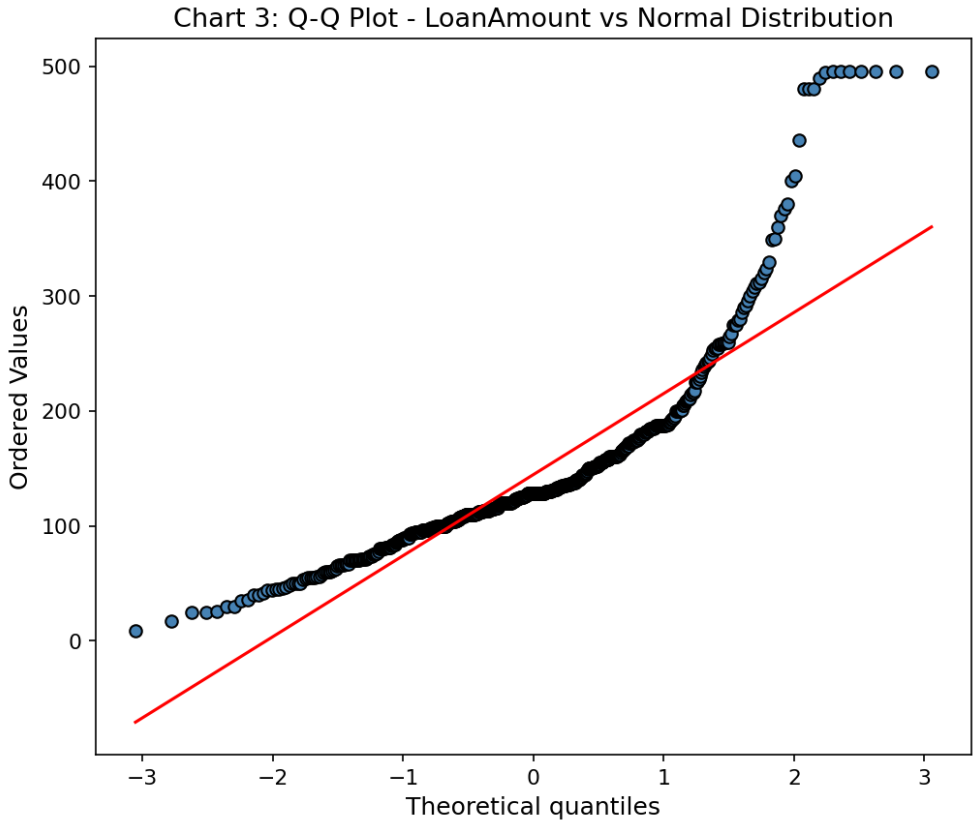


Chart 4: Pie Chart — Overall Loan Approval Outcome

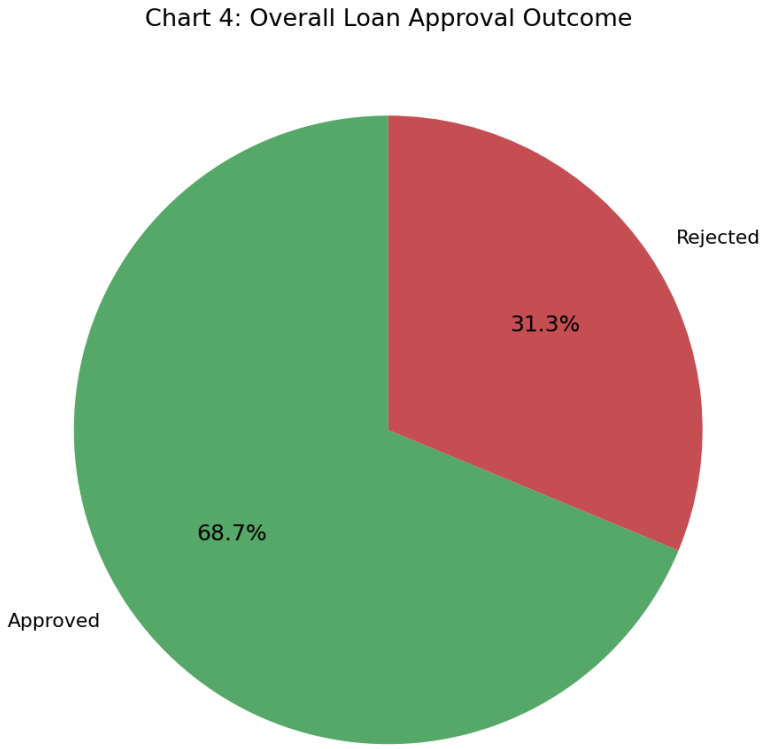


Chart 5: Sunburst — Property Area → Education

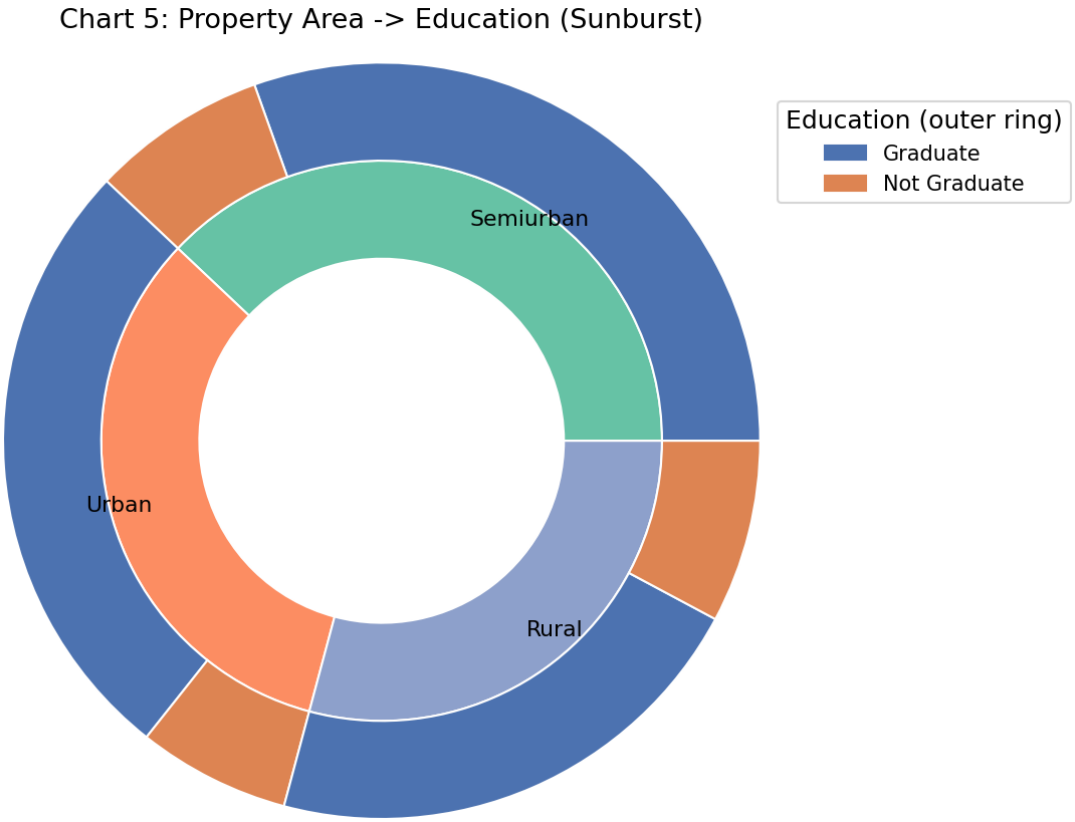
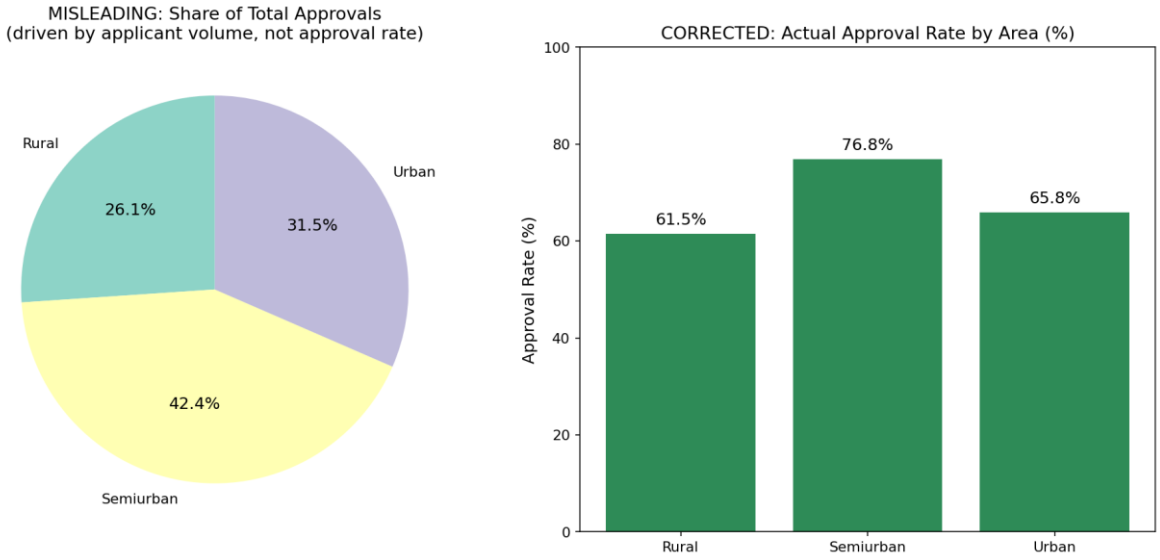


Chart 6: Misleading vs Corrected — Approval by Property Area



Part (c): Interactive Dashboard Design

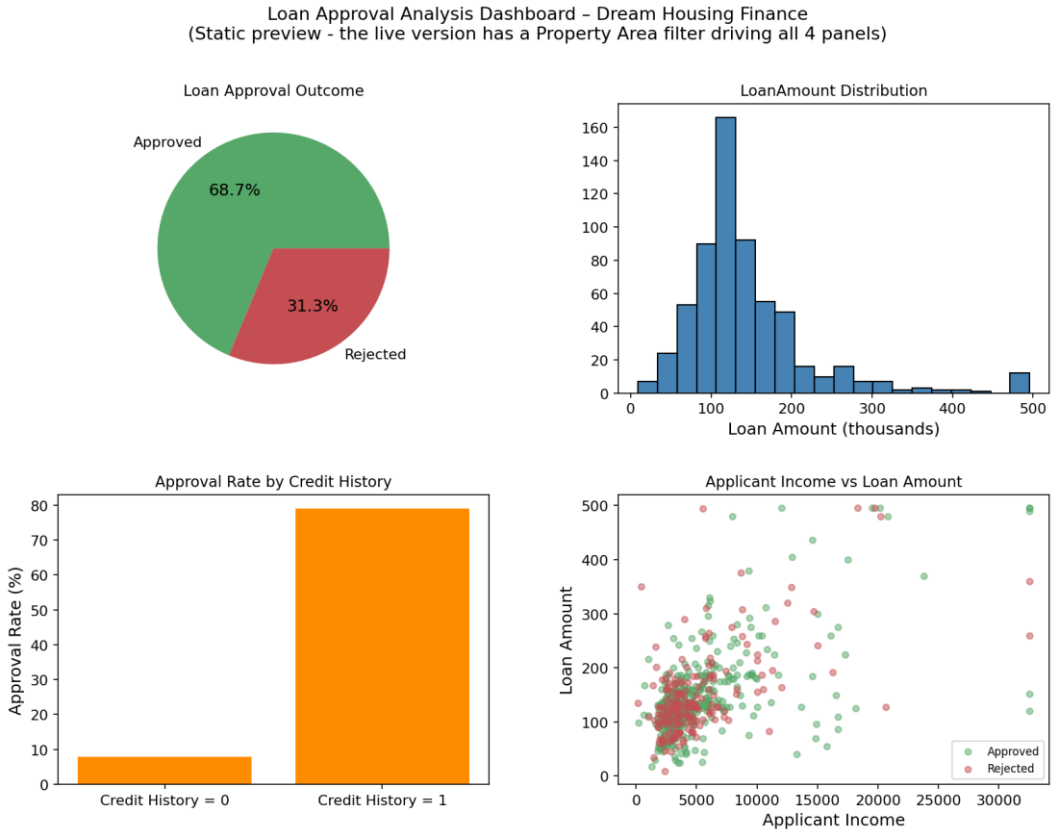
Tool chosen: Plotly (Python), exported as a single self-contained HTML file (loan_dashboard.html, delivered alongside this report). Plotly was chosen over Tableau, Power BI, Dash, and R Shiny specifically because the brief calls for a dashboard a non-technical decision-maker can explore independently: a Plotly HTML export opens with a double-click in any browser, needs no license (Tableau/Power BI), and needs no running server (unlike Dash or R Shiny) — the interactivity is compiled directly into the HTML/JavaScript.

Interactivity: a dropdown filter (top-left of the dashboard) lets the user choose “All Areas”, “Urban”, “Semiurban”, or “Rural”. Selecting an option updates all four panels simultaneously — the approval-outcome pie, the loan-amount histogram, the credit-history approval-rate bars, and the income-vs-loan-amount scatter all re-filter to that property area at once.

```
"""
Part (c): Interactive Dashboard Design
Tool chosen: Plotly (Python), exported as a single self-contained HTML file.

Why Plotly over Tableau / Power BI / Dash / R Shiny for this brief:
- The Head of Credit Risk needs to "explore it independently" - a self-
  contained HTML file opens in any browser with a double-click, with no
  Tableau/Power BI license, no server, and no Dash/Shiny runtime required.
- Plotly's built-in dropdown (updatemenus) provides genuine client-side
  interactivity baked directly into the HTML/JS, so the filter works even
  when the file is emailed or opened offline.
"""
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import pandas as pd

df = pd.read_csv("loan_prediction_cleaned.csv")
areas = ['All Areas'] + sorted(df['Property_Area'].unique().tolist())
status_colors = {'Y': '#55A868', 'N': '#C44E52'}

fig = make_subplots(
    rows=2, cols=2,
    specs=[[{'type': 'domain'}, {'type': 'xy'}],
           [{'type': 'xy'}, {'type': 'xy'}]],
    subplot_titles=("Loan Approval Outcome", "LoanAmount Distribution",
                    "Approval Rate by Credit History", "Applicant Income vs Loan Amount")
)

traces_per_area = 5  # pie(2 slices as 1 trace) + hist(1) + bar(1) + scatter(2: approved/rejected)
n_areas = len(areas)

for area in areas:
    sub = df if area == 'All Areas' else df[df['Property_Area'] == area]
    visible = (area == 'All Areas')

    # Panel 1: pie - approval outcome
    status_counts = sub['Loan_Status'].value_counts()
    fig.add_trace(go.Pie(
        labels=['Approved' if i == 'Y' else 'Rejected' for i in status_counts.index],
        values=status_counts.values,
        marker=dict(colors=[status_colors[i] for i in status_counts.index]),
        visible=visible, name=area, showlegend=visible
    ), row=1, col=1)

    # Panel 2: histogram - LoanAmount
    fig.add_trace(go.Histogram(
        x=sub['LoanAmount'], marker_color='steelblue', visible=visible,
        name=area, showlegend=False
    ), row=1, col=2)

    # Panel 3: bar - approval rate by credit history
    rate_by_ch = sub.groupby('Credit_History')['Loan_Status'].apply(lambda s: (s == 'Y').mean() * 100)
    fig.add_trace(go.Bar(
        x=[f"Credit History = {int(i)}" for i in rate_by_ch.index], y=rate_by_ch.values,
        marker_color='darkorange', visible=visible, name=area, showlegend=False
    ), row=2, col=1)

    # Panel 4: scatter - income vs loan amount, colored by status (2 traces)
    for status, color in status_colors.items():
        s = sub[sub['Loan_Status'] == status]
        fig.add_trace(go.Scatter(
            x=s['ApplicantIncome'], y=s['LoanAmount'], mode='markers',
            marker=dict(color=color, size=6, opacity=0.6),
            name='Approved' if status == 'Y' else 'Rejected',
            visible=visible, showlegend=visible
        ), row=2, col=2)

# Build dropdown: one button per area, toggling visibility of that area's
# block of traces across ALL FOUR panels simultaneously.
buttons = []
for i, area in enumerate(areas):
    vis = [False] * (n_areas * traces_per_area)
    start = i * traces_per_area
    for j in range(traces_per_area):
        vis[start + j] = True
    buttons.append(dict(label=area, method='update',
                        args=[{'visible': vis},
```

```
                                {'annotations[4].text': f"Filter: {area}"}]))

fig.update_layout(
    title=dict(text="Loan Approval Analysis Dashboard \u2014 Dream Housing Finance",
               x=0.5, font=dict(size=20)),
    updatemenus=[dict(active=0, buttons=buttons, x=0.0, y=1.15,
                      xanchor='left', direction='down',
                      pad=dict(t=5, r=5))],
    height=800, width=1100,
    annotations=[
        dict(text="Use the dropdown (top-left) to filter every panel below by Property Area at once.",
             xref='paper', yref='paper', x=0.5, y=1.08, showarrow=False, font=dict(size=12, color='gray')),
        dict(text="Approved vs Rejected split for the selected area.",
             xref='paper', yref='paper', x=0.12, y=1.0, showarrow=False, font=dict(size=9, color='gray')),
        dict(text="Distribution of requested loan amounts (thousands).",
             xref='paper', yref='paper', x=0.88, y=1.0, showarrow=False, font=dict(size=9, color='gray')),
        dict(text="Filter: All Areas",
             xref='paper', yref='paper', x=0.12, y=0.46, showarrow=False, font=dict(size=9, color='gray')),
        dict(text="Credit history is the strongest single predictor of approval.",
             xref='paper', yref='paper', x=0.12, y=0.43, showarrow=False, font=dict(size=9, color='gray')),
        dict(text="Income alone does not clearly separate approved vs rejected.",
             xref='paper', yref='paper', x=0.88, y=0.43, showarrow=False, font=dict(size=9, color='gray')),
    ]
)

fig.write_html("loan_dashboard.html", include_plotlyjs='cdn')
print("Dashboard saved: loan_dashboard.html")
print(f"Total traces: {len(fig.data)} ({n_areas} areas x {traces_per_area} traces per area)")
```

Static Preview ("All Areas" view — open loan_dashboard.html for the live, filterable version)



Part (d): Insights & Recommendations

PART (d): INSIGHTS & RECOMMENDATIONS
Loan Prediction Dataset — Dream Housing Finance
Prepared for: Head of Credit Risk

SUMMARY OF FINDINGS

Across 614 cleaned loan applications, the overall approval rate is 68.7%.
Analysis of every applicant attribute against Loan_Status reveals one overwhelmingly dominant pattern and two secondary patterns worth flagging.

1. CREDIT HISTORY IS THE DOMINANT PREDICTOR, BY A WIDE MARGIN

Applicants with Credit_History = 1 (meets credit guidelines) are approved 79.6% of the time. Applicants with Credit_History = 0 are approved only 7.9% of the time — a roughly 10x difference. No other single attribute in this dataset comes close to this effect size. This is visible directly in Chart 2 (nested donut) and the credit-history panel of the interactive dashboard.

2. INCOME AND LOAN AMOUNT, SURPRISINGLY, SHOW ALMOST NO DIFFERENCE BETWEEN APPROVED AND REJECTED APPLICANTS

Median ApplicantIncome is virtually identical between approved (3,812.5) and rejected (3,833.5) applicants. Median LoanAmount is identical (128.0 for both groups). This means income and requested loan amount, on their own, do NOT meaningfully separate approved from rejected applicants in this dataset — the underwriting outcome is not simply a function of "can this applicant plausibly afford this loan." This is a genuinely counter-intuitive finding worth flagging explicitly, since it means informal income-threshold heuristics would perform poorly as a screening rule here.

3. SECONDARY, MODERATE EFFECTS: EDUCATION, MARITAL STATUS, AND PROPERTY AREA

- Graduates are approved more often than non-graduates (70.8% vs 61.2%).
- Married applicants are approved more often than unmarried applicants (71.8% vs 62.9%).
- Property area shows a real but more modest gap: Semiurban applicants are approved 76.8% of the time, versus 65.8% for Urban and 61.5% for Rural (Chart 6 shows why a naive count-based view of this exaggerates the gap — the corrected, rate-based view is the fair comparison).
- Self-employment status shows almost no effect (68.8% vs 68.3%), and number of dependents shows no clear monotonic pattern.

RECOMMENDATIONS FOR THE HEAD OF CREDIT RISK

1. Treat credit history as the primary automated screening gate, but audit the exceptions. Given its ~10x effect size, credit history should remain the dominant factor in any scoring model or automated pre-screen. However, the data shows this rule is not applied absolutely: 20.4% of Credit_History = 1 applicants were still rejected, and 7.9% of Credit_History = 0 applicants were still approved. These "policy override" cases are worth a targeted manual-underwriting audit to confirm the overrides are well-justified (e.g. strong compensating factors) rather than inconsistent decision-making.

2. Stop relying on income or requested loan amount alone as an informal approval signal. Since median income and median loan amount are nearly identical between approved and rejected applicants, any underwriting guidance that implicitly assumes "higher income or lower loan amount requested = safer applicant" is not well supported by this data. A more useful metric to evaluate going forward would be a debt-to-income or loan-to-income RATIO rather than raw income, since ratios were not directly available as a field here but are a natural next step for the underwriting model.

3. Formally review the property-area gap for legitimacy before it becomes embedded in policy. A ~15 percentage-point gap in approval rate between Semiurban (76.8%) and Rural (61.5%) applicants, independent of the volume-driven distortion shown in Chart 6, deserves a documented policy review: is this gap explained by a legitimate risk factor (e.g. property valuation/collateral liquidity differences), or does it reflect a process inconsistency (e.g. different branch offices, appraisers, or underwriters serving different areas)? This is worth resolving explicitly given fair-lending considerations before the next underwriting-policy refresh.

Part (e): Presentation & Usability

The dashboard (loan_dashboard.html) is titled “Loan Approval Analysis Dashboard – Dream Housing Finance” at the top of the page, so a non-technical viewer immediately knows what they are looking at and which company it concerns.

Each of the 4 panels carries its own short in-dashboard note directly beneath its title (e.g. “Credit history is the strongest single predictor of approval” under the credit-history panel), so the viewer does not need this written report open side-by-side to understand what each chart is showing or why it matters.

The pie chart and scatter plot both carry standard Plotly legends (Approved/Rejected, colour-coded consistently green/red across every panel and every chart in this report), so colour meaning stays consistent whether the viewer is looking at the dashboard or this document.

The dropdown filter itself is labelled with plain-language options (“All Areas”, “Urban”, “Semiurban”, “Rural”) rather than raw column/value names, and an instruction line above it (“Use the dropdown (top-left) to filter every panel below by Property Area at once”) makes the interactivity discoverable without requiring the Head of Credit Risk to guess that the panels are linked.

Submission contents: this document; loan_prediction_raw.csv (original download); loan_prediction_cleaned.csv (cleaned dataset); 01_data_preparation.py, 02_six_charts.py, 03_interactive_dashboard.py (source code); loan_dashboard.html (interactive dashboard); 05_insights_report.txt (plain-text insights).